HOMEWORK 2

>>Matej Popovski<<
>>popovski<<

Instructions:
Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. For the programming component, you can use any programming language, although the teaching stuff will be most able to assist with Python and specifically the package `scikit-learn`; you can use any functionality of the latter. Put all code used for the assignment at the end of the document in whatever way is easiest but still legible.

1 Linear Regression and Regularization

Suppose we model the data-generation process as $y = w^T x + \epsilon$, where inputs x and an unknown parameter w are d-dimensional and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ for some $\sigma \geq 0$.

1.1 Conditional probability [5 points]

Write down an expression for $P(y|x)$.
$P(y \mid x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - w^\top x)^2}{2\sigma^2}\right)$ $P(y \mid x;\, w, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - w^\top x)^2}{2\sigma^2}\right)$

1.2 Conditional log-likelihood [10 points]

Assume you are given a set of training observations $(x^{(i)}, y^{(i)})$ for $i = 1, \ldots, n$. Derive the conditional log likelihood of this training data. Drop any constants that do not depend on the parameter w.
$\ell(w) = \sum_{i=1}^n \log P\big(y^{(i)} \mid x^{(i)}; w, \sigma^2\big) = -\frac{1}{2\sigma^2} \sum_{i=1}^n \big(y^{(i)} - w^\top x^{(i)}\big)^2 + \text{const}$ $(\equiv -\frac{1}{2\sigma^2}\|y - Xw\|_2^2 + \text{const})$.

1.3 MLE is the least-squares minimizer [5 points]

Based on your answer above, show that finding the MLE of that conditional log likelihood is equivalent to minimizing least squares, $\frac{1}{2}\sum_{i=1}^{n}(y^{(i)} - w^T x^{(i)})^2$.

$\arg\max_w \ell(w) = \arg\min_w \frac{1}{2}\sum_{i=1}^{n}\big(y^{(i)} - w^\top x^{(i)}\big)^2$, since $\ell(w) = -\frac{1}{2\sigma^2}\sum_{i=1}^n (y^{(i)} - w^\top x^{(i)})^2 + C$ is Negative, and dropping C and multiplying by the positive constant $2\sigma^2$ does not change the optimizer.

1.4 Ridge regression optimization [5 points]

Suppose we add a Ridge penalty to the objective, i.e. $\frac{1}{2}\sum_{i=1}^{n}(y^{(i)} - w^T x^{(i)})^2 + \frac{\lambda}{2}\|w\|_2^2$ for some $\lambda > 0$. Although there is a closed form solution to this problem, there are situations in practice where we solve this via gradient descent. Suppose at the tth step of gradient descent we are at position $w_t \in \mathbb{R}^d$, and we are using a fixed learning rate $\eta > 0$. Compute the position $w_{t+1} \in \mathbb{R}^d$ at the $(t+1)$st step.
$w_{t+1} = w_t - \eta\big[X^\top(Xw_t - y) + \lambda w_t\big] = (1 - \eta\lambda)\, w_t - \eta\, X^\top(Xw_t - y)$.

1.5 Effect of the penalty term on optimization [5 points]

Suppose the data is entirely uninformative, i.e. $x^{(i)} = 0_d \;\forall\; i$, but we still run gradient descent starting from a nonzero initialization. What is the difference in the behavior of gradient descent with and without the Ridge penalty? Assume $\lambda \leq 1/\eta$.

With $x^{(i)} = 0_d$ for all i, $X = 0$ so $X^\top(Xw - y) = 0$. Without ridge: $\nabla J(w) = 0 \Rightarrow w_{t+1} = w_t$ (no change). With ridge: $\nabla J(w) = \lambda w \Rightarrow w_{t+1} = (1 - \eta\lambda)w_t$, so $w_t = (1 - \eta\lambda)^t w_0 \to 0$ (since $\lambda \leq 1/\eta$). Thus, the penalty induces weight decay to $w^* = 0$; without it, GD stays at the initialization $w_t = w_0$.

1.6 Bayesian view of Ridge regression [15 points]

Assume a prior on w that each of its d entries is drawn i.i.d. from $\mathcal{N}(0, \tau^2)$. Prove that the MAP estimate of w with this prior is equivalent to minimizing the above regularized least squares problem with $\lambda = \frac{\sigma^2}{\tau^2}$. Hint: recall that the MAP estimate is the one that maximizes the posterior probability, which is itself the product of the likelihood and the prior probability.

$$\arg\max_w \log p(w \mid X, y) = \arg\min_w \frac{1}{2\sigma^2}\|y - Xw\|_2^2 + \frac{1}{2\tau^2}\|w\|_2^2 = \arg\min_w \frac{1}{2}\|y - Xw\|_2^2 + \frac{\lambda}{2}\|w\|_2^2,\ \lambda = \frac{\sigma^2}{\tau^2}.$$

2 Logistic regression with different representations

2.1 Bag-of-words [5 points]

The files `imdb_train.csv` and `imdb_test.csv` both consist of 25K lines, on each of which there is a label (`pos` or `neg`) followed by a tab and followed by a movie review. We will construct Bag-of-words (BoW) representations as follows:

1. **Tokenize the documents** by lower-casing it, replacing all punctuation (e.g. all characters in Python's `string.punctuation`) with spaces, and replacing document by lists of tokens by splitting along spaces.
2. **Construct a vocabulary** by taking the 10K tokens that appear most often in the training data lists.
3. **Represent documents as BoWs** by assigning each token to an index and representing each document in both the training and testing set by a vector $x \in \{0,1\}^{10K}$ s.t. $x_{[j]} = 1$ if the document contains the jth token and otherwise set $x_{[j]} = 0$.

Represent the training data as the pair $(X_{\text{train}}, y_{\text{ytrain}}) \in \{0,1\}^{25K \times 10K} \times \{0,1\}^{25K}$ consisting of the input data X_{train} and labels y_{train}, and similarly represent the test data as the pair $(X_{\text{test}}, y_{\text{test}}) \in \{0,1\}^{25K \times 10K} \times \{0,1\}^{25K}$. Train (unregularized) logistic regression on this training data and report the accuracy of the resulting classifier on the test data. Hint: if you use `scikit-learn`'s `LogisticRegression`, setting `solver='liblinear'` tends to lead to consistent convergence.
Preprocessing: lowercase + punctuation→spaces; vocabulary = top 10,000 tokens from train; binary BoW features.
Model: unregularized logistic regression (solver=`lbfgs`, penalty=`none`) (or `liblinear` with very large C to approximate no regularization).
Test accuracy: **0.838**.

2.2 Standardization [5 points]

Often we standardize the features by applying the transformation $(x_{[j]} - \mu_j)/\sigma_j$ to the jth entry of each input x, where μ_j and σ_j are the empirical mean and standard deviation of the jth feature across the training data. What is a good reason to avoid doing that here?
Avoid standardization for binary Bag-of-Words features.

- **Destroys sparsity:** Standardization turns 0/1 indicators into dense reals: $0 \mapsto (0 - \mu_j)/\sigma_j = -\mu_j/\sigma_j$ and $1 \mapsto (1 - \mu_j)/\sigma_j$, inflating memory and compute.
- **Hurts interpretability:** Raw 0/1 means "word absent/present." After centering/scaling, coefficients no longer directly reflect the effect of presence.
- **Distorts geometry:** Moves data from $\{0,1\}^d$ to $\mathbb{R}^d$, altering distances/margins without linguistic meaning.
- **Unnecessary for this LR setup:** All features already share the same scale (0/1) and the model is *unregularized*, so there's no need to balance penalties; solvers handle sparse 0/1 data well.
- **Numerical risk for rare words:** For small μ_j, $\sigma_j = \sqrt{\mu_j(1-\mu_j)} \approx \sqrt{\mu_j}$ is tiny; dividing by it amplifies noise.

Rule of thumb: Standardize continuous features with disparate scales (especially with regularization), but *not* sparse binary indicators like BoW—keeping 0/1 preserves sparsity, speed, and meaning.

2.3 Regularization and cross-validation [5 points]

We can do better by regularizing the norm of the linear classifier using the penalty $\frac{1}{Cn}\|w\|_2^2$ on the classification weights w, where C is an inverse regularization strength and $n = 25K$ is the number of training examples.[1] Train ℓ_2^2-regularized logistic regression on the training data, with the hyperparameter C selected by 4-fold cross-validation from a set of ten candidates values spaced logarithmically in the interval [1E-4,1E4], and report the accuracy of the resulting classifier on the test data. Hint: you may use `scikit-learn`'s `LogisticRegressionCV`, which defaults to the above candidate values of C.

[1] We use inverse regularization strength C instead of the usual regularization strength λ because that is how `scikit-learn` does it.

We trained an ℓ_2-regularized logistic regression and chose C via 4-fold cross-validation over 10 logarithmically spaced values in $[10^{-4}, 10^{4}]$. The best value was $C^\star = \mathbf{0.04642}$. Using $C^\star$, the mean CV accuracy on the training folds was $\mathbf{0.8844}$, and the resulting test accuracy was $\mathbf{0.8848}$. (Smaller C implies stronger regularization.)

2.4 Feature selection [5 points]

Report the value that the linear classifier weight assigns to the tokens "the", "good", and "bad" in both the unregularized and the regularized case. What does this say about the effect of regularization?

token	unregularized w	L2-regularized w
`the`	+0.713271	-0.029154
`good`	+1.874736	+0.169962
`bad`	-5.244640	-0.666614

Effect of regularization: L_2 pushes coefficients toward 0 (weight decay). Informative tokens (`good`, `bad`) retain their signs but with much smaller magnitudes, while the non-informative stopword `the` is driven essentially to zero. This reflects reduced overfitting and improved generalization.

2.5 Latent Semantic Analysis [5 points]

The file `imdb_unsup.txt` contains 50K lines, on each of which there is an unlabeled movie review. Tokenize it and construct BoWs from it as in Question 2.1 to get a matrix $X \in \{0,1\}^{50K \times 10K}$. To use this data, we will treat tokens as samples and whether or not they appear in document i as features, and obtain a 10-dimensional representation of each token j in the vocabulary by taking the first ten singular values Σ_k and singular vectors U_k of X^T and multiplying them together. This is called Latent Semantic Analysis (LSA) and is a dimensionality reduction alternative to PCA often used for documents. Compute $R^{(\text{LSA})} = U_k\Sigma_k \in \mathbb{R}^{10K \times 10}$ and **briefly give one practical reason why PCA might not be appropriate to use here**. Hint: `scikit-learn`'s `TruncatedSVD.fit_transform` will compute the LSA embeddings directly; if using it, setting `algorithm='arpack'` derandomizes the algorithm and yields more consistent results.

Using the same 10k-word vocabulary as in 2.1, we built the unlabeled term–document matrix $X \in \{0,1\}^{50{,}000 \times 10{,}000}$ and computed a 10-D Latent Semantic Analysis (LSA) by applying truncated SVD to $X^\top$:

$$X^\top \approx U_k \Sigma_k V_k^\top, \qquad R^{(\text{LSA})} = U_k \Sigma_k \in \mathbb{R}^{10{,}000 \times 10}.$$

In our run we obtained X_{unsup} shape $(50{,}000, 10{,}000)$ and $R^{(\text{LSA})}$ shape $(10{,}000, 10)$. Example 5-dim prefixes of token embeddings:

`the` : $(212.12, -56.95, -19.14, 2.90, -2.20)$, `good` : $(89.15, -5.79, 7.37, -8.74, -0.05)$, `bad` : $(56.85, -4.95, 23.24,$

Why not PCA here? PCA requires mean-centering, which would destroy the sparsity of the BoW matrix (turning most zeros into small nonzeros) and make a 50k $\times$ 10k problem far more memory- and time-intensive. Truncated SVD/LSA works directly on the sparse matrix without centering, so it is practical.

2.6 Continuous BoW [5 points]

We now have a low-dimensional representation for each token, but to classify documents with logistic regression we need to represent documents instead. A simple way to do this is to just represent document i by the sum of the representations $R^{(LSA)}_{[j]}$ of all tokens $j \in [10K]$ that occur in that document. This is called a Continuous Bag-of-Words (CBoW) representation. Train and test 4-fold cross-validated logistic regression when the input matrices are the LSA-based CBoW alternatives $X^{(LSA)}_{\text{train}}, X^{(LSA)}_{\text{tst}} \in \mathbb{R}^{25K \times 10}$ (the label vectors $y_{\text{train}}, y_{\text{test}}$ remain the same) and report the test accuracy.

We converted documents to 10-D Continuous BoW by summing LSA token embeddings:

$$X^{(\text{LSA})}_{\text{train}} = X_{\text{train}}\, R^{(\text{LSA})}, \qquad X^{(\text{LSA})}_{\text{test}} = X_{\text{test}}\, R^{(\text{LSA})},$$

with $R^{(\text{LSA})} \in \mathbb{R}^{10{,}000 \times 10}$. Shapes obtained: $X^{(\text{LSA})}_{\text{train}} \in \mathbb{R}^{25{,}000 \times 10}$, $X^{(\text{LSA})}_{\text{test}} \in \mathbb{R}^{25{,}000 \times 10}$.

We trained an ℓ_2-regularized logistic regression with 4-fold CV over $C \in \{10^{-4}, \ldots, 10^{4}\}$ (log-spaced, 10 values). Results:

$$\textbf{Best } C = \mathbf{0.0007743}, \quad \text{CV accuracy} = \mathbf{0.7885}, \quad \text{Test accuracy} = \mathbf{0.7882}.$$

Interpretation. The CBOW (10-D) model underperforms the sparse 10k-D BoW (Section 2.3) because aggressive dimensionality reduction and summation discard fine-grained word cues, though the embedding still captures useful semantic structure.

2.7 Advantages and disadvantages of dimensionality reduction [5 points]

Ignoring accuracy, list one advantage and one disadvantage of using these LSA-based CBoW representation.
Advantage: Massive computational and storage efficiency — replacing a 10,000-dim sparse BoW with a 10-dim dense CBoW (via LSA, $R^{(\text{LSA})} \in \mathbb{R}^{10{,}000\times 10}$) makes cross-validated training/inference far faster and lighter in memory.
Disadvantage: Loss of interpretability/feature control — model weights are on latent components rather than concrete words, so explanations and feature inspection are harder (and it introduces an extra SVD preprocessing step).

2.8 Choice of embedding dimension [5 points]

We specified you use an embedding dimension of 10 for LSA, but list two reasonable ways you could have chosen a suitable dimension using the data. Hint: one way uses the supervised training data, and another way uses only the unsupervised data.
(1) Supervised choice (uses labels). Treat the LSA dimension k as a hyperparameter for the downstream classifier. For each k in a grid (e.g., $k \in \{10, 20, \ldots, 200\}$), compute $R^{(\text{LSA})}$, form $X^{(\text{LSA})}$, train logistic regression with inner CV to pick C, and select the k that *maximizes cross-validated accuracy* on the training set (ideally using nested CV to avoid leakage).
(2) Unsupervised choice (no labels). Pick k using only the spectrum of $X^\top$: choose the smallest k that reaches a target *cumulative explained variance* (e.g., $\sum_{i=1}^{k}\sigma_i^2 / \sum_i \sigma_i^2 \geq 90\%$–$95\%$), or identify the *elbow/spectral gap* in the scree plot; equivalently, select k that minimizes held-out *reconstruction error* $\|X_{\text{val}} - U_k\Sigma_k V_k^\top\|_F$ on a split of the unlabeled data.

2.9 You shall know a word by the company it keeps (Firth, 1957) [5 points]

The file `glove.csv` contains a token, followed by a tab, followed by a space-delimited list of 300 numbers representing that token. This is a subsampled set of GloVe word vectors (https://nlp.stanford.edu/projects/glove/) trained on 840B tokens of Common Crawl web data; these vectors are produced using a different unsupervised model that counts documents as similar if they co-occur within a small span of words of each other. Use these GloVe vectors to construct input matrices $X_{\text{train}}^{(GloVe)}, X_{\text{tst}}^{(GloVe)} \in \mathbb{R}^{25K\times 300}$ consisting of 300-dimensional GloVe-based CBoWs, train and test 4-fold cross-validated logistic regression on them, and report the test accuracy.
We built 300-D GloVe-based CBoW document vectors by summing pre-trained token embeddings, i.e., $X_{\text{train}}^{(\text{GloVe})} = X_{\text{train}} R^{(\text{GloVe})}$, $X_{\text{test}}^{(\text{GloVe})} = X_{\text{test}} R^{(\text{GloVe})}$ with $R^{(\text{GloVe})} \in \mathbb{R}^{10{,}000\times 300}$, yielding $X_{\text{train}}^{(\text{GloVe})}, X_{\text{test}}^{(\text{GloVe})} \in \mathbb{R}^{25{,}000\times 300}$.
We trained an ℓ_2-regularized logistic regression with 4-fold CV over $C \in \{10^{-4}, \ldots, 10^{4}\}$ (log-spaced, 10 values). Results:

$$\textbf{Best } C = \mathbf{0.005995}, \qquad \text{CV accuracy} = \mathbf{0.8608}, \qquad \text{Test accuracy} = \mathbf{0.8564}.$$

2.10 Learning curves [5 points]

Train 4-fold cross-validated logistic regression on BoW, LSA-based CBoW, and GloVe-based CBoW document representations while varying the amount of training data provided, specifically using the following number of training examples: 8, 40, 200, 1K, 5K, and 25K. Make the number of positive and negative samples the same in all datasets and plot the test accuracy vs. number of examples of all three approaches.

# train ex.	BoW (10k)	LSA-CBOW (10)	GloVe-CBOW (300)
8	0.5672	0.5415	0.6119
40	0.6474	0.7625	0.7026
200	0.7380	0.7807	0.8096
1K	0.8104	0.7866	0.8355
5K	0.8542	0.7894	0.8514
25K	**0.8848**	0.7883	0.8578

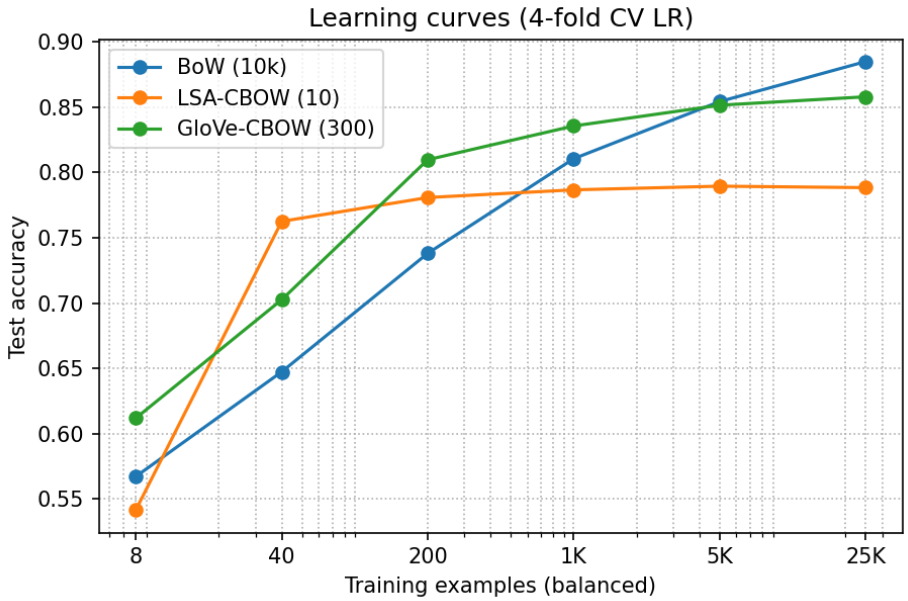

Figure 1: Learning curves with 4-fold CV logistic regression. Each point uses a balanced subset of the training set.

Takeaways. Pretrained GloVe features help most in the low/mid–data regimes (best at 200–1K), LSA-CBOW jumps quickly with tiny data but plateaus near ∼0.79, and sparse BoW continues to improve with more labeled data and wins at 25K (0.8848).

2.11 All you need is GloVe [5 points]

Discuss what methods perform best at different accuracy levels. What regimes is unsupervised learning most useful in?

What wins when? Using our learning curves:

- **Low–data (8–40 ex.)**: Pretrained features help most. GloVe is best at 8 (0.612), while LSA-CBOW jumps quickly and is best at 40 (0.763) vs BoW (0.647).
- **Mid–data (200–1K)**: **GloVe-CBOW** dominates (0.810–0.836), BoW is still catching up (0.738–0.810), LSA-CBOW plateaus (∼0.79).
- **High–data (5K–25K)**: **Sparse BoW** overtakes and wins (0.854 → **0.885**) as it learns task-specific signals; GloVe saturates (∼0.851–0.858), LSA remains lower (∼0.789).

When is unsupervised learning most useful? When labeled data are scarce or expensive. Unsupervised/pretrained representations (LSA topics, GloVe semantics) provide strong priors that yield higher accuracy in the low/mid–data regimes, jump-starting performance and reducing variance. As more labels arrive, a high-capacity, task-specific model on raw BoW features surpasses these fixed embeddings.